# Archive de session — 03/11/2026

Projet Riemann_Lab — session Claude Code du 03/11/2026 (branche Riemann_Lab_IA), reprise après une panne de batterie qui avait coupé la session du matin.

**Turbo Zêta :** RESTAURÉ (vérifié : 4 CPU à leur gouverneur initial (cpu0=powersave), swappiness 10) — première étape du « bye bye ». Un premier essai avec un *sudo* placé devant le script d'enveloppe a échoué (mot de passe demandé) ; la commande correcte, sans *sudo*, a restauré l'état.

**Capture brute :** `claude-traitement-journalier/session_20261103_0946.log` (processus *script* actif au moment du « bye bye »). Le `.md` nettoyé et le `session_*_jsonl.md` seront générés au deuxième *exit*.

Cette archive reprend la narration, les tableaux et les conclusions de la session ; aucun extrait de code n'y figure.

---

Partie A — session du matin (reprise après panne de batterie)

## 1. Point à la coupure

> Il y a eu panne de batterie pendant une session. Lire le dernier session log session_20261103_0747 […]. Fais-moi un point et de là où on était à la coupure.

**État reconstitué à partir du log (statut : mesuré).**

- Le matin : runs V17 à T=20 000 (08h10) et T=200 000 (08h39), puis un run V16 à T=200 000 (09h18) pour comparer V16 et V17 à armes égales.
- Comparaison V16 (nouveau) contre V17 à T=200 000 sur environ 298 000 zéros : 298 028 identiques bit à bit (99,94 %), 172 qui diffèrent, écart maximal $4{,}72 \cdot 10^{-8}$, aucun écart supérieur à $10^{-7}$.
- Conclusion de la session coupée : les écarts observés auparavant viennent surtout du correctif de l'affinage (commit du 01/11), pas de la V17. La V17 n'apporte pas de gain mesurable sur les zéros eux-mêmes à T=200 000 ; son intérêt est la traçabilité (consolidation, journal d'échecs).
- Dernière action avant la coupure : lancement à 09h28 d'un run V17 à T=400 000, mort avec la batterie, sans CSV ni log.
- Git propre côté fichiers suivis ; le PC avait redémarré à 09h43.

## 2. Relance du run T=400 000 et correction des logs de session

> Go, relance le T=400 000. Dans une autre tâche : j'ai vu que les logs de session ont des caractères non lisibles quand j'ouvre le .md. Faire que toute log soit bien lisible toujours.

**Run.** Relancé à 09h54 (PC sur secteur, extinction et veille inhibées). Terminé à 10h24, code retour 0.

**Logs de session (statut : mesuré).**

- Cause : le `.md` produit par `capture.sh` ne retirait pas les séquences d'échappement du terminal de Claude Code (curseur, modes, images).
- Correctif : un nettoyeur dédié retire toutes les séquences d'échappement et refuse d'écrire si un caractère de contrôle subsiste ; `capture.sh` produit désormais trois fichiers (log brut, `.md` nettoyé, transcription lisible du dialogue seul, secrets masqués).
- Les 18 `.md` existants ont été régénérés (aucun caractère d'échappement restant), avec une sauvegarde compressée des anciens.
- Limite connue : le `.md` nettoyé reste fragmenté dans les zones d'interface (redessin du terminal) ; la version vraiment lisible est la transcription issue du fichier de session de Claude Code.
- Le premier test en conditions réelles du nouveau `capture.sh` aura lieu à la fin de cette session.

## 3. Synchronisation des branches

Push sur Riemann_Lab_IA. — Synchronise les autres branches. — Porte aussi les 13 autres commits sur les autres branches.

| Opération | Résultat |
|---|---|
| Correctif des logs (commit 5295764) | poussé sur Riemann_Lab_IA, puis porté par cherry-pick sur main, Riemann_Lab_C, Riemann_Lab_Test |
| « 13 autres commits » | 8 déjà présents (patch identique), 2 déjà appliqués (prompts désuivis et ignorés), 2 PDF volontairement réservés à Riemann_Lab_IA, 1 script de guide public porté |
| Index des runs (commit e5ecf1a) | poussé sur Riemann_Lab_IA et porté sur les trois autres branches |

Aucun push n'a utilisé l'option de force. Les branches avaient divergé : le cherry-pick commit par commit, dans des arbres de travail temporaires (pour ne pas perturber le run en cours), a remplacé tout fast-forward.

## 4. Résultats bruts du run T=400 000

**Scan principal par worker (8 workers).**

| Worker | Zéros | Durée | Affinage C | Repli mpmath | Échecs |
|---|---|---|---|---|---|
| 0 | 80 067 | 573 s | 79 176 | 877 | 0 |
| 1 | 80 067 | 559 s | 79 843 | 224 | 0 |
| 2 | 80 064 | 583 s | 79 913 | 151 | 0 |
| 3 | 80 066 | 627 s | 79 937 | 129 | 0 |
| 4 | 80 066 | 660 s | 79 962 | 104 | 0 |
| 5 | 80 065 | 667 s | 79 992 | 73 | 2 |
| 6 | 80 066 | 672 s | 80 010 | 56 | 0 |
| 7 | 80 065 | 683 s | 80 006 | 59 | 2 |

Score LMFDB : 20 sur 20 à moins de $10^{-9}$. Aucune erreur dans le log. Après un rescan ciblé de 3 segments, validation Turing–Backlund : **640 523 zéros calculés pour 640 526 attendus, soit 3 manquants** (INCOMPLET), répartis ainsi : 2 manquants à T=308 000 et 3 à T=400 000.

## 5. Analyse des 3 zéros manquants

Analyse les 3 zéros manquants.

**Mesuré.** Les trois zéros sont localisés et vérifiés avec la fonction Z de mpmath à 30 chiffres :

| Zéro | t | Z(t) en valeur absolue | Segment / worker |
|---|---|---|---|
| 1 | 286 692,10233507601 | $1{,}9 \cdot 10^{-29}$ | 5 / 5 |
| 2 | 306 086,79828720580 | $3{,}4 \cdot 10^{-29}$ | 5 / 5 |
| 3 | 385 415,72427039186 | $7{,}9 \cdot 10^{-29}$ | 7 / 7 |

- Les deux premiers sont avant T=308 000 (d'où +2 à ce point) ; avec le troisième on obtient +3 à T=400 000 : les deux totaux de Turing sont expliqués exactement.
- Chacun figurait dans le journal d'échecs du run : l'affinage n'avait pas convergé (résidus de $1{,}06 \cdot 10^{-12}$ à $6{,}1 \cdot 10^{-12}$, juste au-dessus de la tolérance $10^{-12}$).
- Dans chacun des trois intervalles de recherche, Z ne change pas de signe : le zéro se trouve à environ $2{,}5 \cdot 10^{-7}$ **à l'extérieur** de l'intervalle. Aucun solveur de racine ne pouvait converger.
- Un quatrième échec (t≈381 076,737) a été récupéré par le rescan.
- Le rescan à pas plus fin n'a pas récupéré ces trois-là ; la raison n'est pas établie.

**Hypothèse (non testée).** Le décalage commun d'environ $2{,}5 \cdot 10^{-7}$ des bords des intervalles pourrait venir de l'erreur en virgule flottante double précision de la détection de signe à t proche de $3 \cdot 10^5$.

## 6. Correction du CSV, Turing relancé, rapport

Go option 1, ajoute les 3 zéros au CSV. — Relance Turing sur le CSV corrigé. — Ajoute une note de correction au rapport. — Régénère le rapport.pdf avec la note.

- Les trois zéros ont été insérés à leur place dans le CSV (marqués d'une méthode d'affinage « manuelle »), la colonne $n$ renumérotée : **640 526 zéros**, ordre strict et numérotation sans trou vérifiés, 3 lignes de différence exactement avec la sauvegarde.
- Turing–Backlund relancé sur le CSV corrigé : **COMPLET**, 0 manquant aux 5 points de contrôle (640 526 calculés pour 640 526 attendus à T=399 999,68). Les points intermédiaires diffèrent du run d'origine car ils sont choisis par rang dans la liste.
- Portée : Turing vérifie un comptage, pas la position de chaque zéro ; les trois zéros ajoutés ont été vérifiés à part.
- Le rapport de monitoring a reçu une note de correction (en section 5, seul canal prévu par le générateur), le PDF a été régénéré. L'alerte « INCOMPLET » en tête de rapport est conservée car elle est lue dans le journal d'exécution du run, volontairement non modifié ; la note l'explique. Le comptage de la section 3 reflète le CSV corrigé.

## 7. Index des runs (Monitoring-Runs.md)

Commite Monitoring-Runs.md. — Option C, restaure à HEAD. — B. — Push e5ecf1a sur Riemann_Lab_IA. — Porte e5ecf1a sur les 3 autres branches.

- Le diff réel n'était pas celui annoncé (une ligne) : la régénération avait reconstruit tout l'index (environ 40 lignes, runs du 24/10 au 03/11), avec une ligne « en cours » périmée pour le run interrompu et des liens vers un dossier non suivi par git. J'ai signalé l'écart au lieu de commiter.
- Choix C (restauration à la version du dépôt), puis choix B : correction à la source du statut du run interrompu (« abandonné, panne de batterie, relancé 09:54 »), régénération de l'index, contrôle, commit, push, puis portage sur les trois autres branches.
- Limite restante : les liens de l'index vers le dossier des runs restent cassés sur GitHub tant que ce dossier n'est pas suivi.

## 8. Question de procédure

> Pour zeta-capture-conv, je fais exit exit avant le bye bye ou le contraire ?

Réponse : « bye bye » d'abord (le turbo et l'archive exigent que Claude tourne encore), puis un premier *exit* pour fermer Claude Code, puis un second pour fermer l'enregistrement, qui génère les fichiers de la capture.

## 9. État de fin de session

- Dépôt : branche Riemann_Lab_IA à jour sur origine (dernier commit e5ecf1a) ; les trois autres branches portent les mêmes correctifs.
- Non commités (volontairement) : archives de session, dossiers de monitoring, sauvegardes, résultats de calcul.
- Run T=400 000 : CSV corrigé de 640 526 zéros, Turing COMPLET, dans le dossier de résultats du run du 03/11 à 09h55.
- Prochaine action possible : décider si le décalage des intervalles de recherche à grand t justifie un correctif du code d'affinage (élargir l'intervalle au lieu d'abandonner sur échec), et si le dossier des runs doit être suivi par git pour réparer les liens de l'index.

---

Partie B — session de 11h58 : panne de `capture.sh` à l'exit, correction, synchronisation

**Turbo Zêta (étape 0 de ce « bye bye ») :** RESTAURÉ (vérifié : 4 CPU à leur gouverneur initial (cpu0=powersave), swappiness 10).

**Capture brute :** `claude-traitement-journalier/session_20261103_1158.log` (processus *script* actif au moment du « bye bye »). Le `.md` nettoyé et le `session_*_jsonl.md` seront générés au deuxième *exit* ; ce sera le premier vrai test de `capture.sh` corrigé.

## 10. Incident de l'exit de la session précédente

> Lors du dernier exit de zeta-capture-conv j'ai eu ceci : erreur de syntaxe ligne 25 de capture.sh. L'affichage du .md semble pas correct, je n'ai pas vu de json comme dit dans la capture d'écran. Fais-moi un point de ce que tu as lu dans la capture et explique les autres problèmes.

**Lecture de la capture d'écran (statut : mesuré).** Elle montre la réponse de la session précédente : run T=400 000 relancé, cause des logs illisibles (séquences d'échappement non retirées par l'ancien nettoyage), trois fichiers promis par session (log brut, `.md` nettoyé, transcription du dialogue), et deux aveux : le `.md` reste fragmenté, et `capture.sh` n'avait pas été testé en conditions réelles.

**Cause de l'erreur ligne 25 (statut : mesuré).** `capture.sh` a été modifié à 09h56 alors que la session démarrée à 09h46 faisait tourner l'ancienne version. Bash lit un script au fil de l'exécution, par positions dans le fichier : le fichier a changé de taille (1 424 → 2 496 octets), donc bash a repris au milieu de la ligne 25 de la nouvelle version. Le fichier lui-même était correct (contrôle de syntaxe réussi). Conséquence : ni nettoyage, ni conversion de la transcription, d'où l'absence du fichier lisible pour la session 0946.

**Autres problèmes relevés.**

- Le `.md` de la session 0946 vient de la régénération de 09h56, pas de la fin normale de la session.
- Mots un par ligne dans le `.md` : le terminal de Claude Code redessine l'écran par morceaux ; le mode de terminal sans couleurs n'y change rien. Seule la transcription du dialogue est fiable.
- Le log de 0946 a continué de grossir jusqu'à 11h51 ; la raison n'a pas été recherchée.

## 11. Correction

Go, fais les deux.

| Action | Résultat |
|---|---|
| Récupération de la session 0946 | transcription lisible générée : `session_20261103_0950_jsonl.md`, 661 lignes, 21 messages de hprzeta, 32 réponses, 71 appels d'outils résumés, 1 secret masqué, contrôle de seconde passe propre |
| Protection de `capture.sh` | tout le corps placé dans une fonction appelée en fin de fichier, suivie d'une sortie immédiate : bash lit la fonction entièrement avant de l'exécuter, donc une modification en cours de session ne peut plus casser la fin |
| Sauvegarde avant modification | copie `capture.sh.bak-20261103b` (locale) |
| Test réel | **non fait** : prévu à l'exit de cette session |

## 12. Commit et synchronisation

Go commit. — Go push. — Go, synchronise les autres branches.

| Branche | Commit | Poussé |
|---|---|---|
| Riemann_Lab_IA | 29d0972 | oui (e5ecf1a..29d0972) |
| main | 8123a44 | oui (4c2b37a..8123a44) |
| Riemann_Lab_C | f75cf8d | oui (4e19804..f75cf8d) |
| Riemann_Lab_Test | 188a206 | oui (6895465..188a206) |

Les historiques des branches divergent depuis la purge des identifiants : report par cherry-pick, sans conflit. Audit des lignes ajoutées avant chaque push : rien trouvé (deux correspondances du filtre étaient de faux positifs). Aucun push n'a utilisé l'option de force. Les branches `inbox-ia` et `session` n'ont pas été touchées.

## 13. État de fin de session (partie B)

- Dépôt : les quatre branches portent le correctif de `capture.sh` ; la branche courante est Riemann_Lab_IA.
- Non commités (volontairement) : archives de session, captures, dossiers de monitoring, sauvegardes.
- Aucun calcul en cours ; le run T=400 000 de la partie A est terminé.
- À vérifier à la prochaine session : que `session_20261103_1158_jsonl.md` a bien été produit à l'exit, et que le message final du script s'est affiché sans erreur.

Partie C — session de 12h17 : élargissement du bracket (V17), suivi de `monitoring/runs/`, outil `zeta-runs-suivre`

**Turbo Zêta (étape 0 de ce « bye bye ») :** RESTAURÉ (vérifié : 4 CPU à leur gouverneur initial (cpu0=powersave), swappiness 10).

**Capture brute :** `claude-traitement-journalier/session_20261103_1217.log`. Attention : le processus *script* de la session 1158 (démarré à 12h15) était encore actif **au-dessus** de celui-ci (deux captures imbriquées). Il faudra plusieurs *exit* pour tout fermer, et le `session_*_jsonl.md` de chaque capture reste à vérifier.

## 14. Point projet

> « point projet »

Bloc de reprise lu dans `~/riemann_handoff/Handoff.md` et affiché tel quel. Résumé en trois lignes : pipeline de production = V17 (validée à T=20 000 et à T=400 000 après ajout manuel de 3 zéros) ; dernier commit `29d0972` (correctif de `capture.sh`), poussé sur les 4 branches ; prochaine action proposée : vérifier `session_20261103_1158_jsonl.md`, élargir le bracket d'affinage, décider du suivi git de `monitoring/runs/`.

## 15. Action 2 — élargir le bracket d'affinage

> « go pour l'action 2 : élargir le bracket »

**Cause (mesurée, pas supposée).** Les 4 échecs journalisés du run T=400 000 ont été rejoués. À $t \approx 3 \cdot 10^5$, la détection *Z_double* (float64) place les bornes du bracket à environ $2{,}5 \cdot 10^{-7}$ du vrai zéro, du mauvais côté. Z calculé par Arb a alors le **même signe** aux deux bornes ; Illinois sort du crochet, le repli *mpmath* lève une erreur de tolérance, et le zéro est perdu. L'hypothèse du Handoff (erreur float64 de la détection) est donc **confirmée** ; Arb n'est pas en cause.

**Correctif.** Une fonction d'élargissement essaie des marges de 0,25, 0,5, 1 puis 2 fois la largeur du bracket de chaque côté, en commençant par la plus petite pour limiter le risque d'englober deux zéros voisins ; elle s'arrête au premier vrai changement de signe de Z (Arb), puis affine. Elle n'est appelée que lorsque le repli *mpmath* échoue ; si aucune marge ne convient, l'échec est journalisé comme avant. Les zéros ainsi retrouvés portent la méthode `arb_C_elargi` dans la provenance et dans les statistiques.

| Bracket en échec (T=400 000) | Résultat avec l'élargissement |
|---|---|
| t ≈ 286 692,1013 | zéro retrouvé : 286 692,102335 |
| t ≈ 306 086,7983 | zéro retrouvé : 306 086,798287 |
| t ≈ 385 415,7233 | zéro retrouvé : 385 415,724270 |

| Bracket en échec (T=400 000) | Résultat avec l'élargissement |
|---|---|
| t ≈ 381 076,7372 | zéro déjà présent via le bracket voisin (doublon, traité par la consolidation ; non rejoué) |

Les trois valeurs retrouvées sont identiques à celles ajoutées à la main dans la partie A. **Tests :** 3 tests unitaires ajoutés ; 28 tests au total, tous réussis. **Limite assumée :** le chemin complet du worker n'a pas été rejoué de bout en bout (un run T=400 000 dure environ 10 h) ; seule la fonction d'élargissement est validée, sur les vrais brackets.

## 16. Commit et synchronisation de l'action 2

« go pour le commit et la sync des 4 branches »

Audit des lignes ajoutées : rien de sensible. Commit `d7fd9ac` sur Riemann_Lab_IA, reporté par cherry-pick sur `main` (fe5d6d8), `Riemann_Lab_C` (a3ddbc3) et `Riemann_Lab_Test` (36651d5) ; tests verts sur chaque branche, aucun push forcé.

## 17. Action 3 — suivi de `monitoring/runs/`

« go pour l'action 3 : suivi de monitoring/runs »

**Constat.** Le dossier pèse 58 Mo ; 30 fichiers étaient déjà suivis. L'examen de ces 30 fichiers a donné la convention en vigueur : on suit les **livrables** (rapport, session, métadonnées, figures) et pas les fichiers de travail (tableurs CSV, fichiers *pid*, journaux, marqueurs *.fin*, sauvegardes). Elle a été reprise telle quelle.

**Audit.** 267 fichiers candidats (environ 36 Mo : 190 en *prod*, 46 en *auto*, 30 en *test*). Texte : aucune IP, MAC, jeton, *duckdns* ni e-mail (seuls des chemins `/home/riemann/...` apparaissent dans les métadonnées). Binaires (PNG, PDF) : de nombreuses « correspondances » du motif e-mail se sont révélées être du bruit de flux compressés ; la recherche ciblée d'IP, de MAC et de *duckdns* n'a rien trouvé. Une première simulation était vide à cause d'une syntaxe d'exclusion incorrecte, corrigée avant tout ajout.

**Résultat.** Commit `3ba9acc` sur Riemann_Lab_IA ; `main` 47a4577, `Riemann_Lab_C` 65ab386, `Riemann_Lab_Test` e8a6bf5. Les liens de l'index `Monitoring-Runs.md` sont tous valides et tous suivis. Le premier cherry-pick sur `Riemann_Lab_Test` a échoué (« le picorage a échoué ») sans rien pousser, puis a réussi au second essai ; **cause non élucidée**.

## 18. Outil `zeta-runs-suivre`

« fait le scrip et ajoute allias et a la lste zeta-scripte »

Un script rend l'opération répétable. Par défaut il **simule** : il liste les fichiers à suivre, leur taille, et lance l'audit, sans rien indexer. Avec l'option d'ajout il indexe, mais seulement si l'audit est propre, et ne commite **jamais** (cohérent avec la règle « pas de commit sans validation »). Testé : cas sans nouveauté, option inconnue (code 2), faux run propre (CSV ignoré), faux run contenant une IP (refus, code 3) ; le faux dossier a été supprimé. L'alias `zeta-runs-suivre` a été ajouté au `~/.bashrc` (sauvegarde `.bak-20261103`) et le menu `zeta-script` le liste en position 41.

« go pour commiter et sync les 4 branches »

Commit `65dc3df` sur Riemann_Lab_IA ; `main` 333ec95, `Riemann_Lab_C` 2edbee6, `Riemann_Lab_Test` 7e7c93f ; les quatre branches sont alignées avec `origin`.

## 19. Mise à jour du Handoff

> « mets à jour le Handoff »

Entrée ajoutée en tête du bloc de reprise de `~/riemann_handoff/Handoff.md` (sauvegarde `.bak11-20261103`). Le Handoff du wiki n'a pas été touché (copie du 03/11 à 08h39).

## 20. État de fin de session (partie C)

- Dépôt : 4 branches alignées avec `origin`, aucun commit en attente ; branche courante Riemann_Lab_IA.
- Non commités (volontairement) : archives de session, captures, sauvegardes `.bak*`.
- Aucun calcul en cours.
- À faire à la prochaine session (rien n'est lancé sans l'accord de hprzeta) : vérifier les `session_*_jsonl.md` produits à l'exit (captures imbriquées) ; valider l'élargissement en conditions réelles (segment d'un worker ou run T=400 000) et mesurer le nombre de zéros `arb_C_elargi` ; lancer `zeta-runs-suivre` après chaque run ; reprendre le bloc du 01/11 (complément $5 \cdot 10^6$, distribution encore sur la V16, tolérance relative, documentation).

---

Partie D — session de l'après-midi (point 2 : validation de l'élargissement du bracket)

## 21. Point projet

> « point projet »

Le bloc de reprise du Handoff local fait 107 Ko ; résumé en 3 lignes : pipeline V17 en production ; dernier commit `65dc3df` ; quatre branches alignées avec `origin`. Points ouverts : valider l'élargissement du bracket en conditions réelles, suivi après chaque run, reste du 01/11.

## 22. Rejeu du segment du worker 5

> « go pour le point 2 : relancer le segment du worker 5 »

Segment `[261070,589 ; 308000,965]` rejoué avec les options `--t-min` et `--t-max`, services arrêtés pendant le run puis redémarrés. Durée 88 s.

| Mesure | Valeur |
|---|---|
| Zéros trouvés | 80 067, identiques au CSV de production corrigé |
| Échecs | 0 |
| Zéros 286 692,102335 et 306 086,798287 | retrouvés par le chemin normal |
| `arb_C_elargi` | 0 : le correctif n'a pas été exercé |

**Conclusion (mesurée) :** ce rejeu ne valide pas le correctif. Le découpage en sous-segments décale la grille de scan, et les brackets ne tombent plus du mauvais côté du zéro. Le 4e point du Handoff restait entier. Trois paires de zéros proches ont été mises en quarantaine AMBIGUOUS par la consolidation.

## 23. Rejeux sur la grille d'origine

« analyse les résultats quand c'est fini » · « donne-moi le résultat du rejeu de grille » · « rejoue le worker 7 » · « rejoue le 4e cas »

Méthode : un seul worker rejoué avec son `t_start` et son `STEP` d'origine (0,000983). Un premier essai a planté au démarrage (constante mal nommée, erreur de ma part), signalé puis corrigé et relancé. Plusieurs moniteurs d'attente ont aussi mal fonctionné : un filtre `pgrep` se détectait lui-même.

| Rejeu | Zéros | Échecs | `arb_C_elargi` | Zéros retrouvés |
|---|---|---|---|---|
| worker 5 jusqu'à 286 700 | 43 573 | 0 | 1 | 286 692,102335 |
| worker 5 jusqu'à 306 100 | 76 800 | 0 | 2 | 286 692 et 306 086,798287 |
| worker 7 jusqu'à 385 420 | 54 442 | 0 | 2 | 381 076,737150 et 385 415,724270 |

**Conclusion (mesurée) :** les 4 échecs du run T=400 000 sont tous rattrapés par l'élargissement du bracket, avec des valeurs identiques à celles ajoutées à la main. Aucun doublon sur 381 076,737150 (écart minimal 0,0153).

## 24. Clôture du point 2 dans le Handoff

« clos le point 2 dans le Handoff » · « go, écris la clôture dans le Handoff »

Attente de la fin du 4e rejeu avant d'écrire, pour ne pas affirmer un résultat non vérifié. Entrée ajoutée en tête du bloc de reprise de `~/riemann_handoff/Handoff.md` (sauvegarde `.bak12-20261103`) : méthode, résultats des 3 rejeux, piège du rejeu par `--t-min`, état, prochaines actions.

## 25. Examen des 3 paires AMBIGUOUS

« examine les 3 paires AMBIGUOUS »

Vérification indépendante avec `mpmath.siegelz` (30 chiffres) et un solveur différent.

| Paire (t autour de) | Écart | Signes de Z avant, entre, après | \|Z\| aux racines |
|---|---|---|---|
| 270 071,29 | 0,0086 | −, +, − | environ 1e-21 |
| 273 193,66 | 0,0057 | −, +, − | environ 1e-21 |
| 302 700,30 | 0,0096 | −, +, − | environ 1e-21 |

**Conclusion (mesurée) :** ce sont de vrais zéros simples et distincts, déjà présents dans le CSV de production. La mise en quarantaine vient de l'absence de `STEP` fourni à la consolidation. **Correction de ma part :** j'avais affirmé à tort, sans vérifier, que ces zéros n'étaient pas dans le CSV d'origine.

« oui, corrige la phrase du Handoff »

Phrase « NON vérifié » remplacée par le résultat vérifié, prochaine action mise à jour (sauvegarde `.bak13-20261103`).

## 26. Commit et mise à jour du wiki

« comit to et met a jour tout » · « confirmez »

Dépôt de code : rien à commiter (sorties de rejeu ignorées par `.gitignore`, `.bak`, `resultats_v17/` et archives laissés non suivis). Wiki : le Handoff local complet n'a PAS été copié (590 lignes, éléments sensibles ou hors périmètre, wiki public) ; section courte « REPRENDRE ICI — 03/11/2026 (après-midi) » ajoutée, audit propre. Commit `2544756`, poussé sur `origin/master` après confirmation, sans `--force`.

## 27. État de fin de session (partie D)

- Point 2 clos : élargissement du bracket validé sur 4 échecs sur 4 en conditions réelles.
- Non fait, à décider par hprzeta : fournir `STEP` à la consolidation ; complément $5 \cdot 10^6$ ; `zeta_distribute` encore sur la V16 ; vérifier les `session_*_jsonl.md` produits à l'exit.
- Non commités (volontairement) : archives de session, captures, sauvegardes `.bak*`, dossier de rejeu `calculs/v17_T261071_308001_20261103_w5/`.
- **Turbo Zêta :** RESTAURÉ (vérifié : 4 CPU à leur gouverneur initial (cpu0=powersave), swappiness 10).
- **Capture brute :** `claude-traitement-journalier/session_20261103_1336.log` (processus *script* actif) ; deux captures imbriquées sont actives (1158 et 1336), donc plusieurs `exit` seront nécessaires.

Partie E — session du soir (STEP, distribution V17, tolérance k = 2)

## 28. Point projet et ordre de travail

« point projet » · « faire les prochaines actions »

Rappel de l'état (pipeline V17, dernier commit `65dc3df`) puis exécution des cinq actions restantes du bloc du 01/11.

## 29. `STEP` fourni à la consolidation

`consolider()` reçoit maintenant `step=STEP` (variante `B_avec_STEP`). Les 3 paires du run T = 400 000 en quarantaine (worker 5, t ≈ 270 071, 273 193 et 302 700 ; écarts 0,0086, 0,0057 et 0,0096) l'étaient uniquement faute de `STEP`. Rejeu T1b : AMBIGUOUS 4 → 0, DISTINCT 0 → 4. 28 tests V17 OK. Gain à T ≥ $10^5$ non mesuré par un run complet.

## 30. `zeta_distribute` passé de la V15 à la V17

Renommage mécanique (script cible, noms de dossiers et de CSV, test d'import PC2) ; `step` du CSV fusionné « adaptatif ». **Jamais exécuté** ; sur PC2, `scripts_v17/` doit être synchronisé avant, sinon le test d'import échoue.

## 31. Tolérance (k)

« k = 2, prépare le patch »

Décision : tolérance de position de 2 ulp. Le patch C préparé pour `illinois_arb.c` est **sans effet mesurable** : 900 zéros réels du run T = 400 000, positions identiques à 0 ulp, temps identiques (le test porte sur le pas de Newton, de l'ordre de $10^{-7}$, pas sur la précision finale). Il n'est **pas appliqué**.

## 32. Où appliquer k = 2 côté validation

> « cherche où appliquer k=2 côté validation »

Un seul endroit utile : la marge de bracket `±1e-10`, qui tombe sous 1 ulp au-delà de t $\approx 4{,}5 \cdot 10^5$. À ne pas toucher : `eps_dup` (32 doublons confirmés sur 8 jeux, **tous d'écart exactement 0**), `tol_doublon`, la vérification LMFDB (20 premiers zéros), `EPS_CONTROLE`. Correction d'un chiffre annoncé en séance : « 49 doublons sur 12 jeux » était faux (rejeux comptés deux fois) ; le bon chiffre est 32 sur 8.

> « go, fais le correctif »

`_marge_bracket(t) = max(1e-10, 2·ulp(t))` aux deux tests de bracket ; inchangée jusqu'à t = 2^18 ; $1{,}86 \cdot 10^{-9}$ à $5 \cdot 10^6$. Tests OK.

## 33. Documentation, audit, commits, cherry-pick, push

> « go, documente dans le Handoff et le JOURNAL » · « go, audit puis commit » · « go, cherry-pick sur les 3 autres branches » · « go, pousse les 4 branches et le wiki »

Handoff local (bloc de reprise), Handoff du wiki et JOURNAL mis à jour (sauvegardes `.bak-20261103e`). Audit des lignes ajoutées : rien. Commit `931a5bf` sur `Riemann_Lab_IA`, porté par `cherry-pick -x` : `46d63c3` (`Riemann_Lab_C`), `bf2b5eb` (`main`), `a8d8077` (`Riemann_Lab_Test`) ; wiki `919bcdd`. Les 4 branches et le wiki sont poussés, sans `--force`.

## 34. État de fin de session (partie E)

- Non fait, à décider par hprzeta : test de `zeta_distribute` V17 (après synchronisation de `scripts_v17/` sur PC2) ; complément $5 \cdot 10^6$ (script de dichotomie sur `N_exact` à écrire d'abord) ; mesure du gain de `STEP` à T $\geq 10^5$ par un run complet.
- **Turbo Zêta :** RESTAURÉ (vérifié : 4 CPU à leur gouverneur initial, `cpu0=powersave`, swappiness 10).
- Non commités (volontairement) : archives de session, captures, sauvegardes `.bak*`, `resultats_v17/`.

Partie F — session de la soirée (test de zeta_distribute V17, complément $5 \cdot 10^6$)

**Turbo Zêta (étape 0 du « bye bye » n° 5) :** RESTAURÉ (vérifié : 4 CPU à leur gouverneur initial (cpu0=powersave), swappiness 10). **Capture brute :** processus *script* actif (`session_20261103_1750.log`) ; un second *script* imbriqué (`session_20261103_1158.log`) est aussi actif : plusieurs *exit* nécessaires.

## 35. Point projet, audit et cherry-pick du commit 931a5bf

> « point projet » · « go audit + commit » · « go cherry-pick et correction du Handoff » · « go push des 3 branches »

Le bloc de reprise annonçait des changements « non commités » : ils l'étaient déjà (commit `931a5bf`, poussé sur *Riemann_Lab_IA*). Audit des lignes ajoutées : rien. Cherry-pick `-x` sur les 3 autres branches, puis push : les commits étaient déjà sur *origin* (cause non identifiée), les 3 branches alignées. Handoff corrigé deux fois (commité, puis poussé).

## 36. Test de zeta_distribute V17

« go test de zeta_distribute V17 » · « go sync PC2 et run T=1000 »

Le dry-run local passe. PC2 n'avait pas la V17 : synchronisés `compute_zeros_v17.py`, `turing_validation.py` (ancien fichier sauvegardé sur PC2) et `scripts_v17/consolider_v17.py`, sommes de contrôle identiques. **Run réel T=1000 :** 649 zéros, Turing COMPLET aux 5 points, 0,14 min. Deux défauts trouvés : le script demande une confirmation interactive (`EOFError` sans terminal) et, surtout, il appelait *zeta_turbo_off* sans l'option des services, ce qui restaurait l'état initial et **coupait le turbo de la session** (INACTIF après le run). Turbo remis à la main ; l'état initial enregistré ensuite est donc celui d'après cet effacement (limite à retenir pour la restauration finale).

## 37. Correctif de zeta_distribute

« go remets le turbo et corrige zeta_distribute » · « go audit + commit + cherry-pick sans push » · « go mets à jour le Handoff puis push des 4 branches »

Appels turbo avec l'option des services (comme *zeta_run.sh*), copie durcie si présente, option `--yes`. Re-testé : T=1000, 649 zéros, Turing COMPLET, turbo ACTIF après. Commit `d4a5bca` ; cherry-pick `e9fe060` (C), `3c23ebd` (Test), `419a5c8` (main) ; 4 branches poussées.

## 38. Complément $5 \cdot 10^6$ : script de dichotomie

« fait la prochaine action » · « audit et commit dabord » · « go cherry-pick et push des 4 branches »

Script `scripts_v17/localiser_manques.py` : dichotomie sur le déficit entre `N_exact` au milieu de chaque intervalle et le nombre de zéros du CSV, puis affinage par balayage de Z et bissection. Test sur T=20 000 avec 4 zéros retirés (dont 2 voisins) : 4 retrouvés sur 4, 48 évaluations, 66 s. Commit `7874e73` ; cherry-pick `b76356c`, `f174460`, `39f10cc` ; poussés.

## 39. Coût de N_exact et run réel

« go mesure le coût de N_exact à $5 \cdot 10^6$ » · « go lance le complément $5 \cdot 10^6$ »

Coût mesuré : 3,5 s par évaluation à vide, 10 à 13 s sous charge (4 workers sur 2 cœurs physiques). Premier lancement arrêté : la sortie redirigée vers un fichier restait en tampon, donc aucune progression visible (et mon premier arrêt a tué par erreur le shell lui-même). Correctif de tampon (commit `c92c202`, cherry-pick `72fb61b`, `0333a3e`, `9b99e49`, poussés) puis relance. **Résultat brut sur le jeu V13 T=$5 \cdot 10^6$ :** déficit total 97 (les 97 manquants du journal), 97 fenêtres d'écart +1, 1 651 évaluations, 3 557 s ; affinage : 97 zéros retrouvés, tous OK, 0 incomplet, 0 doublon, de t = 270 071,30 à 4 958 530,10.

## 40. Analyse, fusion et validation

« go analyse les 97 zéros retrouvés » · « go fusionne les 97 zéros dans une copie du CSV » · « go lance valider_turing sur le fichier fusionné »

Les 97 zéros sont confirmés par mpmath (30 chiffres, changement de signe de Z : 97 sur 97). Distance au plus proche voisin du CSV : 84 à moins de 0,01 (paires supprimées par la déduplication à 0,01) et 13 isolés, tous à $t \leq 1{,}77 \cdot 10^6$. Accord avec le journal du 01/11 (environ 85 paires et 12 isolés). Fusion dans une copie (10 016 474 lignes, triée, 97 lignes marquées, CSV d'origine intact).

**Validation de Turing :** fichier fusionné COMPLET (0 manquant aux 5 points) ; original : 15, 27, 51, 70, puis 97 manquants. Limite : 5 points de contrôle seulement.

## 41. Hypothèse de la grille sur les 13 isolés

> « go teste l'hypothèse de la grille sur les 13 isolés »

Grille de chaque worker V13 reconstruite (celle du worker 0 démarre à t = 65). **Les 13 isolés sont tous à $1{,}1 \cdot 10^{-7}$ à $5{,}7 \cdot 10^{-7}$ d'un point de grille** (test de Kolmogorov-Smirnov contre l'uniforme : p de l'ordre de $10^{-41}$) ; les 84 paires sont uniformes (p = 0,32). Déduction géométrique (non rejouée) : le rescan à STEP/2 contient les points de la grille STEP et ne peut donc pas rattraper ces zéros ; un rescan décalé en phase le pourrait (à décider pour la V17). Mécanisme (signe de Z faux au point de grille, bracket qui n'encadre plus le zéro) : cohérent, non montré sur les 13. Resté inexpliqué : pourquoi tous sont à $t \leq 1{,}77 \cdot 10^6$.

## 42. État de fin de session (partie F)

- Fait, commité et poussé sur les 4 branches : zeta_distribute V17 testé et corrigé, script de dichotomie, correctif de tampon.
- Non commités (volontairement) : tout `calculs/` (jeu fusionné `zeros_v13_T5000000_complete_20261103.csv`, sorties du complément), scripts d'analyse du scratchpad, archives et captures, sauvegardes `.bak*`.
- À décider par hprzeta : sort du jeu fusionné ; rescan décalé en phase pour la V17 ; documentation.
- **Turbo Zêta :** RESTAURÉ (vérifié : 4 CPU à leur gouverneur initial, `cpu0=powersave`, swappiness 10).

---

*discusion_claude_2026-11-03.md — archive de session (parties A, B, C, D, E et F)*